GUJARAT TECHNOLOGICAL UNIVERSITY

Bachelor of Engineering – Semester 4

OPERATING SYSTEMS

Subject Code: BE04000221

Complete Study Notes in Simple English

Feature	Detail
All 9 Chapters	Fully Covered
Algorithms	FCFS, SJF, RR, Priority, Paging, FIFO, LRU, Optimal, SCAN, C-SCAN, SSTF
Practical Programs	C Programs + Shell Scripts
Viva Q&A;	Chapter-wise
Quick Revision	After every chapter
Exam Questions	Short + Long + Numericals

GTU Operating Systems – Complete Study Notes

Subject Code: BE04000221 | Semester 4 | B.E. (Computer / IT)

Simple English | Exam-Oriented | All Topics Covered

TABLE OF CONTENTS

1. Chapter 1 – Introduction to Operating Systems
2. Chapter 2 – Process and Thread Management
3. Chapter 3 – Concurrency & Synchronization
4. Chapter 4 – Inter-Process Communication (IPC)
5. Chapter 5 – Deadlock
6. Chapter 6 – Memory Management
7. Chapter 7 – I/O Management & Disk Scheduling
8. Chapter 8 – Unix/Linux Operating System
9. Chapter 9 – Virtualization Concepts
10. Practical Programs (C Programs + Shell Scripts)
11. Full Quick Revision – All Chapters
12. Important Formulas & Concepts
13. Technical Glossary
14. Most Important Exam Questions

CHAPTER 1 – INTRODUCTION TO OPERATING SYSTEMS

1.1 What is an Operating System?

Definition: An Operating System (OS) is a program (software) that acts as a **middleman** between the user and the computer hardware. It manages all hardware and software resources of the computer.

Simple Example: Think of the OS like a hotel manager. You (user) say "I want food (run a program)." The manager (OS) tells the kitchen (CPU), fridge (Memory), and waiter (I/O) what to do. You don't directly talk to them.

Examples of OS: Windows, Linux, macOS, Android, iOS

1.2 Goals of Operating System

Goal	Meaning
Convenience	Make computer easy to use
Efficiency	Use hardware resources properly
Ability to evolve	Allow updates and new features

1.3 Functions of Operating System

- **Process Management** – Run programs, switch between them
- **Memory Management** – Give RAM to programs, take it back
- **File Management** – Create, delete, read, write files
- **I/O Management** – Handle keyboard, mouse, printer, disk
- **Security** – Protect data from unauthorized access

- **Error Detection** – Find and fix hardware/software errors
- **User Interface** – Give users a way to interact (CLI or GUI)

1.4 Computer System Architecture

```
User Programs
     ↓
Operating System (Kernel)
     ↓
Hardware (CPU, RAM, Disk, I/O)
```

- **Hardware** = Physical parts (CPU, RAM, etc.)
- **Kernel** = Core part of the OS that always runs
- **Shell** = Interface between user and kernel (CLI: bash, zsh)

1.5 System Calls

Definition: A system call is a **request made by a program to the OS** to perform a privileged operation (like reading a file, creating a process).

Think of it like: You (program) can't enter the kitchen (hardware) directly. You write an order slip (system call) and give it to the waiter (OS).

Common System Calls:

Category	Examples
Process Control	fork(), exec(), exit(), wait()
File Management	open(), read(), write(), close()
Device Management	ioctl(), read(), write()
Information	getpid(), time()
Communication	pipe(), socket()

1.6 Types of Operating Systems

1. Batch OS

- Jobs are collected and processed one after another
- No user interaction during execution
- **Example:** Payroll processing

2. Multiprogramming OS

- Multiple programs loaded in memory at the same time
- CPU switches to another job when current job waits for I/O
- **Goal:** Keep CPU busy always

3. Multitasking OS

- Multiple tasks run "simultaneously" by rapid CPU switching
- User feels all tasks are running at same time

- **Example:** Windows 10 – browser + music + typing at once

4. Time-Sharing OS

- Each user/process gets a small time slice (quantum)
- Very fast switching gives feel of simultaneous execution
- **Example:** UNIX system with multiple terminals

5. Parallel OS

- Multiple CPUs work at the same time
- True simultaneous execution
- **Example:** Scientific computing systems

6. Distributed OS

- Multiple computers connected over network act as one system
- **Example:** Google's server clusters

7. Real-Time OS (RTOS)

- Responds to events within a strict time deadline
- **Hard RTOS:** Must meet deadline (pacemaker, aircraft)
- **Soft RTOS:** Missing deadline is acceptable sometimes (video streaming)

1.7 Interaction of OS and Hardware

- OS uses **interrupts** to communicate with hardware
- When hardware (e.g., keyboard) has data ready, it sends an **interrupt signal** to CPU
- CPU stops current task, handles the interrupt, then resumes

Interrupt Types:

- Hardware interrupt (from devices)
- Software interrupt (from programs – system calls)
- Timer interrupt (for time-sharing)

■ QUICK REVISION – CHAPTER 1

Topic	Key Point
OS Definition	Middleman between user and hardware
Kernel	Core of OS, always running
System Call	Program's request to OS for service
Batch OS	Jobs run one after another, no interaction
Multiprogramming	Multiple jobs in RAM, CPU never idle
Multitasking	Rapid switching gives illusion of parallel run
Time-Sharing	Each user gets fixed time slice
Real-Time OS	Must respond within strict deadlines

Memory Trick: "**B**ig **M**achines **M**ake **T**he **P**eople **D**o **R**eal things"

= Batch, Multiprogramming, Multitasking, Time-sharing, Parallel, Distributed, Real-time

CHAPTER 2 – PROCESS AND THREAD MANAGEMENT

2.1 What is a Process?

Definition: A **process** is a **program in execution**. When you run a program (like Chrome), it becomes a process.

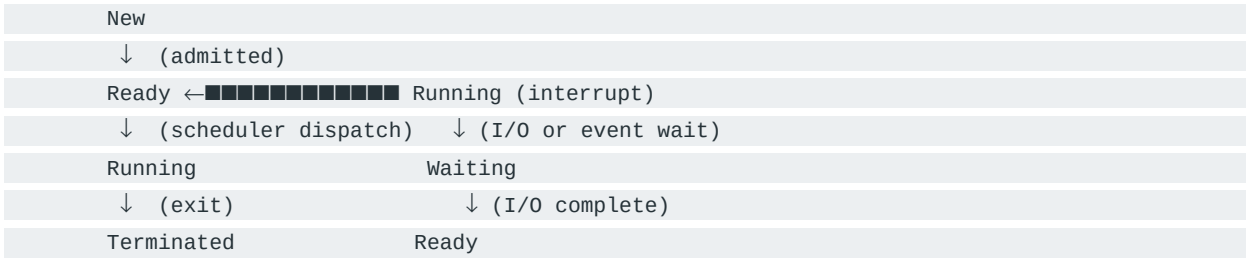
- **Program** = A static file on disk (recipe on paper)
- **Process** = Running program in memory (cooking in kitchen)

A process contains:

- Program code (instructions)
- Program counter (which instruction is next)
- Stack (temporary data, function calls)
- Heap (dynamically allocated memory)
- Data section (global variables)

2.2 Process States

A process goes through different states:



State	Meaning
New	Process is being created
Ready	Process is waiting in queue for CPU
Running	Process is currently using CPU
Waiting (Blocked)	Process is waiting for I/O or event
Terminated	Process has finished

Memory Trick: "**N**aughty **R**abbits **R**un **W**ildly **T**oday" = New, Ready, Running, Waiting, Terminated

2.3 Process Control Block (PCB)

Definition: PCB is a data structure that stores all information about a process.

PCB Contains:

- Process ID (PID)
- Process State

- Program Counter
- CPU Registers
- Memory Management Info
- I/O Status Info
- Accounting Info (CPU time used)

Think of PCB as: A student's file in school (stores all details about that student)

2.4 CPU Scheduling

Definition: CPU scheduling decides **which process gets CPU next** and for how long.

Types of Scheduling:

Type	Meaning
Preemptive	OS can stop (preempt) a running process and give CPU to another
Non-Preemptive	Process keeps CPU until it finishes or voluntarily releases

Scheduling Terms:

Term	Definition
Arrival Time (AT)	When process arrives in ready queue
Burst Time (BT)	Time needed to complete the process
Completion Time (CT)	When process finishes
Turnaround Time (TAT)	CT – AT (total time from arrival to finish)
Waiting Time (WT)	TAT – BT (time process waited in queue)
Response Time	Time from arrival to first CPU access

2.5 Scheduling Algorithms

A) FCFS – First Come First Serve

Rule: Process that arrives first gets CPU first. Like a queue at a bank.

Type: Non-Preemptive

Example:

Process	AT	BT
P1	0	4
P2	1	3
P3	2	1

Gantt Chart:

```
| P1 (0-4) | P2 (4-7) | P3 (7-8) |
```

Process	AT	BT	CT	TAT	WT
P1	0	4	4	4	0
P2	1	3	7	6	3
P3	2	1	8	6	5

- Avg TAT = (4+6+6)/3 = **5.33**
- Avg WT = (0+3+5)/3 = **2.67**

Disadvantage: Convoy effect – short process waits behind long process

---

B) SJF – Shortest Job First

Rule: Process with **shortest burst time** gets CPU next.

Type: Can be Preemptive (SRTF) or Non-Preemptive

Example (Non-Preemptive):

Process	AT	BT
P1	0	6
P2	1	4
P3	2	2

Steps:

- At t=0: Only P1 available → P1 runs (0 to 6)
- At t=6: P2 (BT=4) and P3 (BT=2) available → P3 runs (6 to 8)
- At t=8: P2 runs (8 to 12)

Gantt Chart:

```
| P1 (0-6) | P3 (6-8) | P2 (8-12) |
```

Advantage: Minimum average waiting time

Disadvantage: Starvation of long processes; burst time may not be known

---

C) Round Robin (RR)

Rule: Each process gets a **fixed time slice (quantum)**. If not done, goes back to end of queue.

Type: Preemptive

Example (Quantum = 2):

Process	AT	BT
P1	0	5
P2	1	3
P3	2	1

Gantt Chart (step-by-step):

```
Time 0-2: P1 runs (remaining BT=3)
Time 2-4: P2 runs (remaining BT=1)
Time 4-5: P3 runs (remaining BT=0) → P3 done
Time 5-7: P1 runs (remaining BT=1)
```

```
Time 7-8: P2 runs (remaining BT=0) → P2 done
Time 8-9: P1 runs → P1 done
|P1|P2|P3|P1|P2|P1|
0  2  4  5  7  8  9
```

Advantage: Fair, no starvation

Disadvantage: High context switching overhead

D) Priority Scheduling

Rule: Process with **highest priority** (usually smallest number = highest priority) gets CPU.

Type: Preemptive or Non-Preemptive

Example:

Process	AT	BT	Priority
P1	0	4	3
P2	0	2	1
P3	0	3	2

Order: P2 (priority 1) → P3 (priority 2) → P1 (priority 3)

Gantt Chart:

```
| P2 (0-2) | P3 (2-5) | P1 (5-9) |
```

Problem: Starvation – low priority processes never get CPU

Solution: Aging – gradually increase priority of waiting processes

---

2.6 Real-Time Scheduling

Used in: RTOS (pacemaker, aircraft, missile systems)

Algorithm	Description
Rate Monotonic (RM)	Higher frequency task = higher priority
Earliest Deadline First (EDF)	Task with nearest deadline runs first

---

2.7 Threads

Definition: A thread is the **smallest unit of CPU execution** within a process.

Analogy: Process = Restaurant. Threads = Individual waiters. One restaurant (process) can have many waiters (threads) working simultaneously.

Why threads?

- Threads share the same memory space (same process)
- Lighter than processes (less overhead to create)
- Better performance for concurrent tasks

Thread vs Process:

Feature	Process	Thread
Memory	Separate	Shared
Creation time	Slow	Fast
Communication	Complex (IPC needed)	Easy (shared memory)
Context switch	Slow	Fast

Types of Threads:

- **User-level threads** – Managed by user-space library, OS doesn't know about them
- **Kernel-level threads** – Managed directly by OS

2.8 System Calls for Processes in Linux

```
fork()   // Create child process (exact copy of parent)
exec()   // Replace process with new program
wait()   // Parent waits for child to finish
exit()   // Terminate process
getpid() // Get current process ID
```

fork() Return Values:

- Returns 0 to child process
- Returns child's PID to parent
- Returns -1 on failure

■ QUICK REVISION – CHAPTER 2

Algorithm	Type	Key Feature
FCFS	Non-preemptive	First come first serve, simple
SJF	Non-preemptive	Min avg WT, starvation possible
RR	Preemptive	Quantum-based, fair, no starvation
Priority	Both	Based on priority, aging fixes starvation

Formulas:

- TAT = CT – AT
- WT = TAT – BT
- Avg WT = Sum of all WT / Number of processes

CHAPTER 3 – CONCURRENCY & SYNCHRONIZATION

3.1 What is Concurrency?

Definition: Concurrency means **multiple processes/threads running at the same time** (or appearing to).

Problem: When multiple processes access **shared data** at the same time, data can become inconsistent.

Example: Two people withdrawing money from the same bank account simultaneously can cause problems (one transaction gets lost).

3.2 Race Condition

Definition: A situation where the result of a computation depends on the **order** in which processes execute.

Example:

```
Account balance = 1000
Process A reads balance = 1000, adds 500
Process B reads balance = 1000, subtracts 200
A writes 1500, B writes 800
Final balance = 800 (WRONG! Should be 1300)
```

3.3 Critical Section

Definition: A **critical section** is a part of code that accesses shared resources. Only ONE process should execute it at a time.

Requirements for Critical Section Solution:

1. **Mutual Exclusion** – Only one process in critical section at a time
2. **Progress** – If no process is in CS, one waiting should enter soon
3. **Bounded Waiting** – A process should not wait forever (finite waiting)

3.4 Mutual Exclusion – Software Approaches

Peterson's Solution (for 2 processes)

```
// Shared variables:
int turn;
boolean flag[2];  // flag[i] = true means Process i wants to enter CS

// Process i:
flag[i] = true;
turn = j;               // Give chance to other process
while (flag[j] && turn == j); // Wait if other wants and it's their turn
// --- Critical Section ---
flag[i] = false;
// --- Remainder Section ---
```

3.5 Hardware Support for Mutual Exclusion

Test-and-Set Instruction

- Atomic hardware instruction (cannot be interrupted)
- Reads and sets a value in one step

```
boolean TestAndSet(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;
}
```

3.6 Semaphores

Definition: A semaphore is an **integer variable** used for synchronization. Access only through two atomic operations: **wait (P)** and **signal (V)**.

```
wait(S):   while S <= 0; S--;    // Decrease (lock)
signal(S): S++;                   // Increase (unlock)
```

Types:

Type	Value Range	Use
Binary Semaphore	0 or 1	Mutual exclusion (like mutex)
Counting Semaphore	0 to N	Manage pool of resources

Producer-Consumer with Semaphore:

```
semaphore mutex = 1;   // Mutual exclusion
semaphore empty = N;   // Number of empty slots
semaphore full = 0;    // Number of filled slots

// Producer:
wait(empty);
wait(mutex);
// produce item, add to buffer
signal(mutex);
signal(full);

// Consumer:
wait(full);
wait(mutex);
// remove item from buffer
signal(mutex);
signal(empty);
```

3.7 Monitors

Definition: A monitor is a **high-level synchronization construct** – a class where only one method can execute at a time.

- Automatic mutual exclusion (unlike semaphores, you don't manually lock/unlock)
- Easier to use, less error-prone
- Used in Java (`synchronized` keyword)

3.8 Pipes

Definition: A pipe is a **one-way communication channel** between processes.

```
ls | grep ".txt"   # Output of ls sent as input to grep
```

- Unidirectional (data flows one way)
- Parent and child process can use pipes

3.9 Message Passing

Definition: Processes communicate by **sending and receiving messages**.

```
send(destination, message)
receive(source, message)
```

Types:

- **Direct** – Name the process explicitly
- **Indirect** – Use mailboxes/ports

3.10 Signals

Definition: Signals are **software interrupts** sent to a process.

Signal	Meaning
SIGINT	Ctrl+C (terminate)
SIGKILL	Kill immediately
SIGSEGV	Segmentation fault
SIGALRM	Timer alarm

■ QUICK REVISION – CHAPTER 3

Concept	Key Point
Race Condition	Result depends on execution order
Critical Section	Code accessing shared resource
Mutual Exclusion	Only one in CS at a time
Semaphore	Integer variable, wait/signal operations
Monitor	High-level sync, auto mutual exclusion
Pipe	One-way communication between processes

CHAPTER 4 – INTER-PROCESS COMMUNICATION (IPC)

4.1 What is IPC?

Definition: Inter-Process Communication (IPC) is a set of **mechanisms for processes to exchange data and synchronize** with each other.

Why needed? Processes normally have separate memory. IPC allows them to share data.

IPC Mechanisms:

1. Shared Memory
2. Message Passing
3. Pipes
4. Semaphores
5. Signals
6. Sockets

4.2 Shared Memory

- A region of memory is shared between processes
- Fastest IPC method (no kernel involved after setup)
- Requires synchronization (semaphores) to avoid race conditions

```
// Create shared memory
int shmid = shmget(key, size, IPC_CREAT | 0666);
// Attach to process
char *str = (char*) shmat(shmid, NULL, 0);
// Use str...
// Detach
shmdt(str);
```

4.3 Classical IPC Problems

A) Producer-Consumer Problem

Scenario:

- Producer creates data and puts in buffer
- Consumer takes data from buffer
- Buffer size is limited (bounded buffer)

Problem: Producer must not add if buffer full. Consumer must not remove if buffer empty.

Solution: Use semaphores (see Chapter 3)

B) Reader-Writer Problem

Scenario:

- Multiple readers can read at same time
- Writer needs exclusive access (no readers/writers allowed)

Solution:

```
semaphore mutex = 1, wrt = 1;
int readcount = 0;

```

```
// Reader:
wait(mutex);
readcount++;
if (readcount == 1) wait(wrt);  // First reader blocks writer
signal(mutex);
// READ
wait(mutex);
readcount--;
if (readcount == 0) signal(wrt);  // Last reader releases writer
signal(mutex);

// Writer:
wait(wrt);
// WRITE
signal(wrt);
```

C) Dining Philosopher Problem

Scenario:

- 5 philosophers sit at a round table
- 5 forks (one between each pair)
- Philosopher needs 2 forks to eat
- Think, pick up left fork, pick up right fork, eat, put down forks

Problem: All 5 pick up left fork → Deadlock! (all waiting for right fork)

Solutions:

- Allow at most 4 philosophers to sit at once
- Pick up both forks or none (atomic)
- Odd philosopher picks left first, even picks right first

■ QUICK REVISION – CHAPTER 4

Problem	Key Concept
Producer-Consumer	Bounded buffer, semaphores for sync
Reader-Writer	Multiple readers OK, writer needs exclusive access
Dining Philosopher	Circular wait → deadlock, various solutions

---

CHAPTER 5 – DEADLOCK

5.1 What is Deadlock?

Definition: Deadlock is a situation where a **set of processes are permanently blocked**, each waiting for a resource held by another process in the set.

Simple Example:

- Process A holds File, needs Printer
- Process B holds Printer, needs File
- Both wait forever → Deadlock!

5.2 Conditions for Deadlock (Coffman's 4 Conditions)

ALL 4 must be present for deadlock:

Condition	Meaning
Mutual Exclusion	Resource can be used by only one process at a time
Hold and Wait	Process holds a resource and waits for another
No Preemption	Resource cannot be forcibly taken from a process
Circular Wait	P1 waits for P2, P2 waits for P3, P3 waits for P1

Memory Trick: "**M**y **H**ouse **N**eeds **C**leaning" = Mutual exclusion, Hold & wait, No preemption, Circular wait

---

5.3 Resource Allocation Graph (RAG)

- **Circle** = Process
- **Square** = Resource
- **Edge from Process** → **Resource** = Process requesting resource
- **Edge from Resource** → **Process** = Resource allocated to process
- **Cycle in RAG** = Deadlock (if single instance per resource)

5.4 Deadlock Prevention

Break at least one of the 4 conditions:

Condition to Break	Method
Mutual Exclusion	Make resources sharable (not always possible)
Hold and Wait	Process must request ALL resources at start
No Preemption	Forcibly take resource if needed
Circular Wait	Order all resources, request in order only

5.5 Deadlock Avoidance

Idea: Before granting a resource, check if it leads to a safe state.

Banker's Algorithm

Concept: Like a banker who only gives loans if he can still give remaining customers their maximum needed amount.

Key Terms:

- **Available** – Currently available instances of each resource
- **Max** – Maximum need of each process
- **Allocation** – Currently allocated to each process
- **Need** = Max – Allocation

Safety Algorithm Steps:

1. Find a process whose Need ≤ Available
2. Allocate resources, let it run, then release all its resources
3. Update Available
4. Repeat until all processes finish → **Safe State**
5. If no such process found → **Unsafe State**

Example:

3 Resources (A=10, B=5, C=7), 5 processes

Process	Max (A,B,C)	Alloc (A,B,C)	Need (A,B,C)
P0	7,5,3	0,1,0	7,4,3
P1	3,2,2	2,0,0	1,2,2
P2	9,0,2	3,0,2	6,0,0
P3	2,2,2	2,1,1	0,1,1
P4	4,3,3	0,0,2	4,3,1

Available = (3,3,2)

Safe sequence: P1→P3→P4→P2→P0 ✓

5.6 Deadlock Detection

When deadlock prevention/avoidance not used, detect deadlock when it occurs.

Method:

- Use Resource Allocation Graph
- Find cycles in the graph
- If cycle found with single instance → Deadlock

5.7 Deadlock Recovery

Once deadlock detected, recover by:

1. **Process Termination** – Kill one or more deadlocked processes
2. **Resource Preemption** – Take resource from one process, give to another

---

5.8 Starvation

Definition: A process waits indefinitely because other processes always get the resource first.

Difference from Deadlock:

- Deadlock: Multiple processes permanently blocked

• Starvation: One process keeps waiting (others make progress)

Solution: Aging – gradually increase priority of waiting processes

---

❑ QUICK REVISION – CHAPTER 5

Concept	Key Point
Deadlock	All 4 Coffman conditions must hold
Prevention	Break one of 4 conditions
Avoidance	Banker's Algorithm – check safe state before granting
Detection	Check RAG for cycles
Recovery	Kill process or preempt resource
Starvation	Process waits, not blocked; fix with aging

CHAPTER 6 – MEMORY MANAGEMENT

6.1 Why Memory Management?

• RAM is limited. Many processes need it.
• OS must allocate memory efficiently, protect processes from each other.

6.2 Memory Management Requirements

1. **Relocation** – Process can be loaded at any memory location
2. **Protection** – Process cannot access another's memory
3. **Sharing** – Allow multiple processes to share memory (e.g., shared library)
4. **Logical Organization** – Manage code, data, stack separately
5. **Physical Organization** – Manage actual RAM efficiently

6.3 Fixed Partitioning

Method: RAM is divided into **fixed-size partitions** at boot time.

```
| OS (0-100KB) | P1 (100-200KB) | P2 (200-300KB) | P3 (300-400KB) |
```

Advantage: Simple

Disadvantage: Internal fragmentation (process smaller than partition – wasted space inside)

6.4 Variable Partitioning

Method: Partitions created **dynamically** based on process size.

Advantage: No internal fragmentation

Disadvantage: External fragmentation (free holes between processes)

6.5 Memory Allocation Strategies

Given: Multiple free holes in memory. Where to place new process?

Strategy	Method	Advantage	Disadvantage
First Fit	Place in first hole that fits	Fast	May waste beginning of memory
Best Fit	Place in smallest hole that fits	Less wasted space	Slow, creates tiny unusable holes
Worst Fit	Place in largest hole	Leaves large holes	Not very useful in practice

Example: Free holes: 100KB, 500KB, 200KB, 300KB. Process needs 250KB.

- First Fit → 500KB hole (first one big enough)
- Best Fit → 300KB hole (smallest that fits)
- Worst Fit → 500KB hole (largest)

6.6 Fragmentation

Type	Description	Occurs In
Internal	Wasted space INSIDE a partition	Fixed partitioning
External	Free memory exists but NOT contiguous	Variable partitioning

Compaction: Shuffle processes in memory to combine free holes. Expensive operation.

6.7 Swapping

Definition: Moving a process from RAM to disk (swap space) temporarily to free up RAM.

```
RAM ←→ Swap Space (on Disk)
```

- Process moved out = **swapped out**
- Process moved in = **swapped in**
- Allows more processes than RAM can hold at once

6.8 Paging

Definition: Paging divides **physical memory into fixed frames** and **logical memory into fixed pages**.

- **Page** = Fixed-size block of logical memory
- **Frame** = Fixed-size block of physical memory (same size as page)
- Pages are loaded into any available frames (non-contiguous)

Page Table: Maps logical page number → physical frame number

Address Translation:

```
Logical Address = (Page Number, Offset)
Physical Address = (Frame Number, Offset)

Frame Number = Page Table[Page Number]
Physical Address = Frame Number × Page Size + Offset
```

Example:

- Page size = 4KB, Page table: [0→2, 1→5, 2→3]
- Logical address: Page 1, Offset 100
- Physical address: Frame 5, Offset 100 = 5×4096 + 100 = 20580

Advantage: No external fragmentation

Disadvantage: Internal fragmentation (last page may not be full)

6.9 Virtual Memory

Definition: Virtual memory allows a process to use **more memory than physically available** by keeping some of it on disk.

- Only the **active parts** of a process are loaded in RAM
- Rest stays on disk in **swap space**
- **Page fault** occurs when needed page is not in RAM

6.10 Demand Paging

Definition: Pages are loaded into RAM **only when needed** (on demand).

Steps when page fault occurs:

1. OS detects page fault (page not in RAM)
2. OS finds free frame in RAM
3. OS loads required page from disk
4. Update page table
5. Resume process execution

6.11 Page Replacement Algorithms

When RAM is full and new page needed, which existing page to replace?

A) FIFO – First In First Out

Rule: Replace the page that has been in memory **longest** (oldest page).

Example:

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

3 frames available

```
Page | Frames     | Fault?
1    | 1 - -      | Yes
```

```
2    | 1 2 -      | Yes
3    | 1 2 3      | Yes
4    | 4 2 3      | Yes (replace 1, oldest)
1    | 4 1 3      | Yes (replace 2)
2    | 4 1 2      | Yes (replace 3)
5    | 5 1 2      | Yes (replace 4)
1    |            | No (1 in memory)
2    |            | No
3    | 5 3 2      | Yes (replace 1)
4    | 5 3 4      | Yes (replace 2)
5    |            | No
```

Total page faults = 9

Disadvantage: Belady's Anomaly – more frames may cause more faults!

B) LRU – Least Recently Used

Rule: Replace the page that was **used least recently** (hasn't been used for longest time).

Example: Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (3 frames)

```
Page | Frames (recent→oldest) | Fault?
1    | 1                      | Yes
2    | 2 1                    | Yes
3    | 3 2 1                  | Yes
4    | 4 3 2 (replace 1-LRU)  | Yes
1    | 1 4 3 (replace 2-LRU)  | Yes
2    | 2 1 4 (replace 3-LRU)  | Yes
5    | 5 2 1 (replace 4-LRU)  | Yes
1    | 1 5 2                  | No
2    | 2 1 5                  | No
3    | 3 2 1 (replace 5-LRU)  | Yes
4    | 4 3 2 (replace 1-LRU)  | Yes
5    | 5 4 3 (replace 2-LRU)  | Yes
```

Total page faults = 10

Best in practice – approximates optimal behavior

C) Optimal Page Replacement

Rule: Replace the page that **will not be used for the longest time** in the future.

Advantage: Gives minimum page faults (theoretical best)

Disadvantage: Requires future knowledge – impossible in real systems (used only for comparison)

Example: Reference: 1,2,3,4,1,2,5,1,2,3,4,5 (3 frames)

```
Page | Decision | Frames | Fault?
1    |          | 1      | Yes
2    |          | 1 2    | Yes
3    |          | 1 2 3  | Yes
4    | Replace 3 (used farthest in future: 3 at pos 10, 1 at pos 4, 2 at pos 5) | 1 2 4 | Yes
1    |          | 1 2 4  | No
```

```
2    |          | 1 2 4  | No
5    | Replace 4 (not used again soon) | 1 2 5 | Yes
...
```

Total page faults = 6 (minimum)

6.12 Thrashing

Definition: Thrashing occurs when OS spends **more time paging (swapping pages)** than executing actual processes.

Cause: Too many processes, not enough frames per process.

Effect: CPU utilization drops drastically (lots of page faults)

Solution: Reduce the number of active processes (working set model)

■ QUICK REVISION – CHAPTER 6

Concept	Key Point
Internal Fragmentation	Wasted space inside partition
External Fragmentation	Free holes not contiguous
Paging	Page table maps logical→physical
Demand Paging	Load page only when needed
FIFO	Replace oldest page
LRU	Replace least recently used
Optimal	Replace page used farthest in future
Thrashing	Too many page faults, CPU busy swapping

Page Fault Rate: Fewer frames → more faults (generally)

CHAPTER 7 – I/O MANAGEMENT & DISK SCHEDULING

7.1 I/O Devices

Types:

Type	Examples
Block devices	Disk, USB (data in blocks, random access)
Character devices	Keyboard, mouse, serial port (data stream)
Network devices	Network cards

7.2 I/O Buffering

Definition: Buffering means **storing data temporarily** while it's being transferred between processes and devices.

Why? Device speed ≠ Process speed. Buffer absorbs the difference.

Types:

- **Single buffering** – One buffer (alternate fill/empty)
- **Double buffering** – Two buffers (fill one while emptying other)
- **Circular buffering** – Ring of buffers for streaming

7.3 Disk Structure

- Disk has **platters** → **tracks** → **sectors**
- **Cylinder** = Same track on all platters
- **Seek time** = Time to move head to correct track (most expensive!)
- **Rotational latency** = Time for disk to rotate to correct sector
- **Transfer time** = Time to transfer data

7.4 Disk Scheduling Algorithms

Goal: Minimize seek time (head movement).

Initial Head Position for all examples below: **Head at 50**

Disk cylinders: 0 to 199

Request queue: 98, 183, 37, 122, 14, 124, 65, 67

A) FCFS – First Come First Serve

Rule: Serve requests in **arrival order**.

```
Head: 50 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
```

Total head movement = |50-98|+|98-183|+|183-37|+|37-122|+|122-14|+|14-124|+|124-65|+|65-67|

= 48+85+146+85+108+110+59+2 = **643 cylinders**

Disadvantage: High head movement

B) SSTF – Shortest Seek Time First

Rule: Serve the request **closest to current head position** next.

```
Head: 50
Sorted by distance: 37(13), 65(15), 67(17), 98(48)...
Order: 50→65→67→37→14→98→122→124→183
```

Total = 15+2+30+23+84+24+2+59 = **239 cylinders**

Advantage: Less movement than FCFS

Disadvantage: Starvation of far requests

C) SCAN (Elevator Algorithm)

Rule: Head moves in **one direction**, serving all requests in that direction. When reaching end (or last request in direction), reverses.

```
Head at 50, moving toward 199:
50 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14
```

Total movement = (199-50) + (199-14) = 149 + 185 = **334 cylinders**

Advantage: More uniform service than SSTF

D) C-SCAN – Circular SCAN

Rule: Head moves in **one direction only**, serving requests. When reaching end, jumps back to beginning **without serving** in return trip.

```
Head at 50, moving toward 199:
50 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → (jump to 0) → 14 → 37
```

Total = (199-50) + (199-0) + (37-0) = 149 + 199 + 37 = **385 cylinders**

Advantage: More uniform waiting time than SCAN

7.5 Comparison of Disk Algorithms

Algorithm	Movement	Starvation?	Use Case
FCFS	Highest	No	Light load
SSTF	Low	Yes	General
SCAN	Medium	No	Heavy load
C-SCAN	Medium	No	Uniform service needed

7.6 RAID

RAID = Redundant Array of Independent Disks

Purpose: Improve disk performance and/or reliability by using multiple disks.

RAID Level	Method	Feature
RAID 0	Striping	Speed, no redundancy
RAID 1	Mirroring	Full copy on second disk
RAID 5	Striping + Parity	Balance of speed & reliability

■ QUICK REVISION – CHAPTER 7

Algorithm	Key Rule
FCFS	Serve in arrival order
SSTF	Serve nearest request

Algorithm	Key Rule
SCAN	Go in one direction, reverse at end
C-SCAN	Go in one direction, jump back to start

CHAPTER 8 – UNIX/LINUX OPERATING SYSTEM

8.1 Linux Introduction

- Linux = Open-source OS, kernel written by Linus Torvalds (1991)
- UNIX was the original OS (Bell Labs, 1969)
- Linux is a UNIX-like OS

Linux Architecture:

```
User → Shell → System Calls → Kernel → Hardware
```

8.2 Basic Linux Commands

Command	Syntax	Purpose
`ls`	`ls -l`	List files
`cd`	`cd /home`	Change directory
`pwd`	`pwd`	Print working directory
`mkdir`	`mkdir dir1`	Create directory
`rm`	`rm file.txt`	Remove file
`rmdir`	`rmdir dir1`	Remove empty directory
`cp`	`cp src dest`	Copy file
`mv`	`mv src dest`	Move/rename file
`cat`	`cat file.txt`	Display file content
`echo`	`echo "Hello"`	Print text
`man`	`man ls`	Show manual page
`chmod`	`chmod 755 file`	Change permissions
`chown`	`chown user file`	Change owner
`ps`	`ps -aux`	Show running processes
`kill`	`kill PID`	Kill a process
`grep`	`grep "word" file`	Search text in file
`wc`	`wc -l file`	Word/line count
`sort`	`sort file`	Sort file contents
`head`	`head -5 file`	First 5 lines
`tail`	`tail -5 file`	Last 5 lines
`find`	`find . -name "*.txt"`	Find files

Command	Syntax	Purpose
`who`	`who`	Show logged in users
`date`	`date`	Show current date/time
`df`	`df -h`	Disk usage
`du`	`du -s dir`	Directory size
`tar`	`tar -cvf file.tar dir`	Create archive
`ping`	`ping google.com`	Network test

8.3 File Permissions

Format: `rwxrwxrwx` (owner-group-others)

Symbol	Meaning	Value
r	Read	4
w	Write	2
x	Execute	1
-	No permission	0

Example: `chmod 754 file`

- Owner: 7 = rwx (read+write+execute)
- Group: 5 = r-x (read+execute)
- Others: 4 = r-- (read only)

8.4 Linux Directory Structure

```
/              (Root)
■■■ bin         (Essential commands: ls, cat, etc.)
■■■ etc         (Configuration files)
■■■ home        (User home directories)
■■■ var         (Variable data: logs, mail)
■■■ tmp         (Temporary files)
■■■ usr         (User programs)
■■■ dev         (Device files)
■■■ proc        (Process information, virtual filesystem)
■■■ lib         (Shared libraries)
■■■ root        (Root user home)
```

8.5 Shell Programming

Shell = Command interpreter. Types: bash, sh, zsh, csh

Shell Script Basics:

```
#!/bin/bash          # Shebang line – tells OS which shell to use
# This is a comment
```

```
# Variables
name="GTU"
echo "Hello $name"

# Read input
echo "Enter number:"
read num

# Arithmetic
sum=$((5 + 3))
echo "Sum = $sum"
```

If-Else:

```
if [ $num -gt 0 ]; then
    echo "Positive"
elif [ $num -lt 0 ]; then
    echo "Negative"
else
    echo "Zero"
fi
```

Comparison Operators:

Operator	Meaning
`-eq`	Equal
`-ne`	Not equal
`-gt`	Greater than
`-lt`	Less than
`-ge`	Greater or equal
`-le`	Less or equal

For Loop:

```
for i in 1 2 3 4 5; do
    echo "Number: $i"
done

# Range
for i in $(seq 1 5); do
    echo $i
done
```

While Loop:

```
i=1
while [ $i -le 5 ]; do
    echo $i
    i=$((i+1))
done
```

Functions:

```
greet() {
    echo "Hello $1"    # $1 is first argument
}
greet "GTU"
```

8.6 Filters and Advanced Commands

Command	Use
`grep -i`	Case-insensitive search
`grep -v`	Invert match (show non-matching lines)
`sed 's/old/new/g'`	Replace text in file
`awk '{print $1}'`	Print first column
`cut -d':' -f1`	Cut by delimiter
`tr 'a-z' 'A-Z'`	Translate lowercase to uppercase
`uniq`	Remove duplicate lines
`tee`	Read from stdin, write to file AND stdout

■ QUICK REVISION – CHAPTER 8

Category	Commands
Navigation	ls, cd, pwd
File ops	cp, mv, rm, mkdir
Viewing	cat, head, tail, grep
Permissions	chmod, chown
Process	ps, kill, top
Archive	tar, gzip

Shell Script: Must start with `#!/bin/bash`

CHAPTER 9 – VIRTUALIZATION CONCEPTS

9.1 What is Virtualization?

Definition: Virtualization is the technology that allows **multiple operating systems to run simultaneously on a single physical hardware**.

Think of it like: One computer acting as many computers.

Example: A single server running Windows Server AND Linux Ubuntu AND macOS at the same time.

9.2 Why Virtualization?

- Better use of expensive hardware
- Run multiple OS on one machine
- Easy to create/delete/backup VMs
- Testing environments
- Cloud computing (AWS, Azure use virtualization)

9.3 Key Terms

Term	Meaning
Host OS	The actual OS installed on physical hardware
Guest OS	The OS running inside the virtual machine
Hypervisor	Software that creates and manages VMs
Virtual Machine (VM)	Simulated computer running on physical hardware

---

9.4 Types of Hypervisors

Type 1 – Bare Metal (True/Pure Virtualization)

- Runs **directly on hardware** (no host OS)
- More efficient
- Examples: **VMware ESXi, Microsoft Hyper-V, Xen**

```
Hardware
    ↓
Hypervisor (Type 1)
    ↓
VM1   VM2   VM3
```

Type 2 – Hosted Virtualization

- Runs **on top of host OS**
- Less efficient, easier to set up
- Examples: **VirtualBox, VMware Workstation**

```
Hardware
    ↓
Host OS
    ↓
Hypervisor (Type 2)
    ↓
VM1   VM2
```

9.5 Approaches to Virtualization

Processor Virtualization

- Multiple VMs share one CPU

- Hypervisor manages CPU time for each VM

Memory Virtualization

- Each VM thinks it has its own RAM
- Hypervisor maps virtual memory to actual physical memory

I/O Virtualization

- VMs share physical I/O devices
- Hypervisor routes I/O requests to appropriate VM

9.6 Specific Virtualization Technologies

Technology	Type	OS
VMware ESXi	Type 1 Hypervisor	Cross-platform
Microsoft Hyper-V	Type 1 Hypervisor	Windows
Xen	Type 1 Hypervisor	Linux-based
Java VM (JVM)	Language VM	Cross-platform
Linux VServer	OS-level virtualization	Linux
Android VM (ART)	App runtime	Android

9.7 Java Virtual Machine (JVM)

- Java code compiles to **bytecode** (not machine code)
- JVM interprets bytecode on any platform
- **"Write Once, Run Anywhere"**

```
Java Code → Bytecode (.class) → JVM → Machine Instructions
```

9.8 Android Virtual Machine

- Android uses **ART (Android Runtime)** (previously Dalvik VM)
- Apps compiled to run on ART, which runs on Linux kernel
- Allows Android apps to work on many hardware types

■ QUICK REVISION – CHAPTER 9

Concept	Key Point
Virtualization	Multiple OS on one hardware
Type 1 Hypervisor	On bare hardware (ESXi, Hyper-V)
Type 2 Hypervisor	On host OS (VirtualBox)
JVM	Runs Java bytecode anywhere
Benefits	Resource sharing, isolation, easy backup

PRACTICAL PROGRAMS

Shell Scripts

Practical 2.2 – Sum and Factorial

```
#!/bin/bash
echo "Enter a number:"
read n

# Sum from 1 to n
sum=0
i=1
while [ $i -le $n ]; do
    sum=$((sum + i))
    i=$((i + 1))
done
echo "Sum from 1 to $n = $sum"

# Factorial
fact=1
i=1
while [ $i -le $n ]; do
    fact=$((fact * i))
    i=$((i + 1))
done
echo "Factorial of $n = $fact"
```

Practical 2.3 – Prime Number Check

```
#!/bin/bash
echo "Enter a number:"
read n

if [ $n -le 1 ]; then
    echo "$n is NOT prime"
    exit
fi

is_prime=1
i=2
while [ $i -le $((n/2)) ]; do
    if [ $((n % i)) -eq 0 ]; then
        is_prime=0
        break
    fi
    i=$((i+1))
done
```

```
if [ $is_prime -eq 1 ]; then
    echo "$n is PRIME"
else
    echo "$n is NOT prime"
fi
```

Practical 3.1 – Palindrome Check

```
#!/bin/bash
echo "Enter a string:"
read str
rev=$(echo "$str" | rev)
if [ "$str" = "$rev" ]; then
    echo "$str is a PALINDROME"
else
    echo "$str is NOT a palindrome"
fi
```

Practical 3.2 – Sum of Digits

```
#!/bin/bash
echo "Enter a number:"
read n
sum=0
temp=$n
while [ $temp -gt 0 ]; do
    digit=$((temp % 10))
    sum=$((sum + digit))
    temp=$((temp / 10))
done
echo "Sum of digits of $n = $sum"
```

Practical 3.3 – Sort N Numbers (Descending)

```
#!/bin/bash
echo "Enter numbers as arguments: ./script.sh 5 3 8 1 9"
arr=("$@")
n=${#arr[@]}

# Bubble sort descending
for ((i=0; i<n-1; i++)); do
    for ((j=0; j<n-i-1; j++)); do
        if [ ${arr[j]} -lt ${arr[j+1]} ]; then
            temp=${arr[j]}
            arr[j]=${arr[j+1]}
            arr[j+1]=$temp
        fi
    done
done

echo "Sorted (Descending): ${arr[@]}"
```

Practical 7.1 – Max, Min, Average

```
#!/bin/bash
echo "Enter numbers separated by spaces:"
read -a nums
n=${#nums[@]}

max=${nums[0]}
min=${nums[0]}
sum=0

for num in "${nums[@]}"; do
    sum=$((sum + num))
    [ $num -gt $max ] && max=$num
    [ $num -lt $min ] && min=$num
done

avg=$(echo "scale=2; $sum/$n" | bc)
echo "Max=$max, Min=$min, Average=$avg"
```

Practical 8.2 – 5 Largest Files

```
#!/bin/bash
ls -lS | head -6 | tail -5
# -S sorts by size (largest first)
# head -6 skips the "total" line plus gets 5 files
```

Practical 8.3 – Count Files in Directory

```
#!/bin/bash
echo "Enter directory path:"
read dir
count=$(ls -l "$dir" | grep -v ^d | tail -n +2 | wc -l)
echo "Number of files in $dir = $count"
```

Practical 10.1 – Translate to Uppercase

```
#!/bin/bash
echo "Enter filename:"
read filename
tr 'a-z' 'A-Z' < "$filename"
```

Practical 10.2 – Change Extension .txt to .doc

```
#!/bin/bash
for file in *.txt; do
    mv "$file" "${file%.txt}.doc"
    echo "Renamed $file to ${file%.txt}.doc"
done
```

Practical 10.3 – Compare Two Files

```
#!/bin/bash
echo "Enter first filename:"
read f1
echo "Enter second filename:"
read f2

if diff -q "$f1" "$f2" > /dev/null 2>&1; then
    echo "Files are SAME"
else
    echo "Files are DIFFERENT"
fi
```

C Programs

Practical 4.1 – Create N Child Processes with fork()

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int n, i;
    printf("Enter number of child processes to create: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            printf("Fork failed!\n");
            exit(1);
        } else if (pid == 0) {
            // Child process
            printf("Child %d: PID=%d, Parent PID=%d\n", i+1, getpid(), getppid());
            exit(0);  // Child exits after printing
        }
        // Parent continues the loop
    }

    // Parent waits for all children
    for (i = 0; i < n; i++) {
        wait(NULL);
    }

    printf("Parent: All children done. Parent PID=%d\n", getpid());
    return 0;
}
```

Practical 4.3 – fork() and exec() to run 'who' and 'ls'

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    // First child runs 'who'
    pid1 = fork();
    if (pid1 == 0) {
        printf("Child 1 (PID=%d) running 'who':\n", getpid());
        execlp("who", "who", NULL);
        exit(0);
    }

    // Second child runs 'ls'
    pid2 = fork();
    if (pid2 == 0) {
        printf("Child 2 (PID=%d) running 'ls':\n", getpid());
        execlp("ls", "ls", "-l", NULL);
        exit(0);
    }

    // Parent waits for both children
    wait(NULL);
    wait(NULL);
    printf("Parent: Both children finished.\n");

    return 0;
}
```

Practical 5.1 – Two Threads (Addition & Subtraction)

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define SIZE 5
int A[] = {1, 2, 3, 4, 5};
int B[] = {5, 4, 3, 2, 1};
int add_result[SIZE], sub_result[SIZE];

void* add_arrays(void* arg) {
    for (int i = 0; i < SIZE; i++) {
        add_result[i] = A[i] + B[i];
    }
    printf("Addition done by Thread 1\n");
    return NULL;
}

void* sub_arrays(void* arg) {
```

```
    for (int i = 0; i < SIZE; i++) {
        sub_result[i] = A[i] - B[i];
    }
    printf("Subtraction done by Thread 2\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, add_arrays, NULL);
    pthread_create(&t2, NULL, sub_arrays, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Addition results: ");
    for (int i = 0; i < SIZE; i++) printf("%d ", add_result[i]);
    printf("\nSubtraction results: ");
    for (int i = 0; i < SIZE; i++) printf("%d ", sub_result[i]);
    printf("\n");

    return 0;
}
// Compile: gcc -o prog prog.c -lpthread
```

Practical 6.1 – Shared Memory

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#include <unistd.h>

int main() {
    key_t key = 1234;
    int shmid;
    char *shm_ptr;

    // Create shared memory
    shmid = shmget(key, 100, IPC_CREAT | 0666);
    if (shmid < 0) { perror("shmget"); exit(1); }

    pid_t pid = fork();

    if (pid == 0) {
        // Child: Read from shared memory
        sleep(1);  // Wait for parent to write
        shm_ptr = (char*)shmat(shmid, NULL, 0);
        printf("Child read: %s\n", shm_ptr);
        shmdt(shm_ptr);
```

```
    } else {
        // Parent: Write to shared memory
        shm_ptr = (char*)shmat(shmid, NULL, 0);
        strcpy(shm_ptr, "Hello from Parent!");
        printf("Parent wrote to shared memory\n");
        shmdt(shm_ptr);
        wait(NULL);
        shmctl(shmid, IPC_RMID, NULL);  // Remove shared memory
    }

    return 0;
}
```

Practical 6.2 – Semaphores in Linux

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <unistd.h>

union semun { int val; };

void sem_wait(int semid) {
    struct sembuf op = {0, -1, 0};  // Decrease by 1
    semop(semid, &op, 1);
}

void sem_signal(int semid) {
    struct sembuf op = {0, 1, 0};  // Increase by 1
    semop(semid, &op, 1);
}

int main() {
    key_t key = 5678;
    int semid = semget(key, 1, IPC_CREAT | 0666);
    union semun arg;
    arg.val = 1;
    semctl(semid, 0, SETVAL, arg);  // Initialize semaphore to 1

    pid_t pid = fork();

    if (pid == 0) {
        // Child
        sem_wait(semid);
        printf("Child in critical section\n");
        sleep(2);
        printf("Child leaving critical section\n");
        sem_signal(semid);
    } else {
        // Parent
        sleep(1);  // Let child enter first
```

```
        sem_wait(semid);
        printf("Parent in critical section\n");
        sem_signal(semid);
        wait(NULL);
        semctl(semid, 0, IPC_RMID);
    }

    return 0;
}
```

Practical 9.1 – Implement 'ls' command

```
#include <stdio.h>
#include <dirent.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    char *path = ".";
    if (argc > 1) path = argv[1];

    DIR *dir = opendir(path);
    if (!dir) { perror("opendir"); exit(1); }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_name[0] != '.')  // Skip hidden files
            printf("%s\n", entry->d_name);
    }

    closedir(dir);
    return 0;
}
```

Practical 9.2 – Implement 'who' command

```
#include <stdio.h>
#include <stdlib.h>
#include <utmp.h>
#include <time.h>

int main() {
    struct utmp *entry;

    setutent();  // Open utmp file

    while ((entry = getutent()) != NULL) {
        if (entry->ut_type == USER_PROCESS) {
            time_t t = entry->ut_time;
            char *time_str = ctime(&t);
            time_str[24] = '\0';  // Remove newline
            printf("%-10s %-15s %s\n",
                   entry->ut_user,
```

```
                    entry->ut_line,
                    time_str);
        }
    }

    endutent();  // Close utmp file
    return 0;
}
```

VIVA QUESTIONS AND ANSWERS

Chapter 1 – Introduction

Q1. What is an Operating System?

A: Software that manages hardware resources and provides services to programs. Acts as interface between user and hardware.

Q2. What is a kernel?

A: The core part of the OS that always remains in memory. Handles low-level tasks (process, memory, I/O).

Q3. Difference between multitasking and multiprogramming?

A: Multiprogramming = multiple jobs in memory, CPU switches when job waits for I/O. Multitasking = CPU switches rapidly between processes giving illusion of simultaneous execution.

Q4. What is a system call?

A: A request by a process to the OS for a privileged service like reading a file or creating a process.

Chapter 2 – Process & Scheduling

Q5. What is the difference between program and process?

A: Program = static code on disk. Process = program in execution (active, in memory).

Q6. What is PCB?

A: Process Control Block – data structure storing all information about a process (PID, state, registers, memory info).

Q7. Which scheduling algorithm has minimum average waiting time?

A: SJF (Shortest Job First) – proven theoretically optimal for average waiting time.

Q8. What is Belady's Anomaly?

A: In FIFO page replacement, adding more frames sometimes causes MORE page faults (not fewer).

Q9. What is context switching?

A: Saving the state of current process and loading the state of the next process. Overhead of multitasking.

Chapter 3 – Synchronization

Q10. What are the 3 requirements of critical section solution?

A: Mutual Exclusion, Progress, Bounded Waiting.

Q11. What is a semaphore?

A: Integer variable for synchronization, accessed only through wait() and signal() atomic operations.

Q12. Difference between binary and counting semaphore?

A: Binary semaphore: 0 or 1 (mutex). Counting semaphore: any non-negative integer (manage N resources).

Q13. What is a monitor?

A: High-level synchronization construct – a class where only one procedure can execute at a time. Built-in mutual exclusion.

Chapter 5 – Deadlock

Q14. What are Coffman's 4 conditions for deadlock?

A: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

Q15. What is Banker's Algorithm?

A: Deadlock avoidance algorithm that checks if granting a resource leads to a safe state before actually granting it.

Q16. Difference between deadlock and starvation?

A: Deadlock: group of processes permanently blocked. Starvation: one process waits indefinitely while others progress.

Chapter 6 – Memory

Q17. What is paging?

A: Dividing logical memory into pages and physical memory into frames. Pages loaded into any available frames (non-contiguous).

Q18. What is thrashing?

A: When OS spends more time swapping pages than executing processes. Caused by insufficient frames.

Q19. What is virtual memory?

A: Technique allowing processes to use more memory than physically available by keeping unused parts on disk.

Q20. Which page replacement algorithm is optimal?

A: Optimal algorithm gives minimum page faults but is theoretical (requires future knowledge).

Chapter 7 – I/O & Disk

Q21. What is Belady's Anomaly?

A: In FIFO, sometimes more frames cause more page faults. Does NOT occur in LRU and Optimal.

Q22. Which disk scheduling minimizes seek time on average?

A: SSTF (Shortest Seek Time First) gives minimum average seek time but can cause starvation.

Q23. What is RAID?

A: Redundant Array of Independent Disks. Uses multiple disks for performance (RAID 0) or reliability (RAID 1) or both (RAID 5).

Chapter 8 – Linux

Q24. What is a shell?

A: Command interpreter between user and kernel. Examples: bash, sh, zsh.

Q25. What does fork() return?

A: 0 to child process, child's PID to parent, -1 on failure.

Q26. What is the purpose of exec()?

A: Replaces current process image with a new program.

Chapter 9 – Virtualization

Q27. Difference between Type 1 and Type 2 hypervisor?

A: Type 1 runs directly on hardware (ESXi, Hyper-V). Type 2 runs on host OS (VirtualBox, VMware Workstation).

Q28. What is JVM?

A: Java Virtual Machine. Runs Java bytecode on any platform – enables "write once, run anywhere."

FULL SYLLABUS QUICK REVISION

Chapter-wise One-Page Summary

Chapter	Key Topics	Most Important Points
1. Intro	OS types, system calls	Batch, Multiprogramming, Multitasking, RT OS
2. Process	States, scheduling	FCFS/SJF/RR/Priority, TAT=CT-AT, WT=TAT-BT
3. Sync	Semaphore, mutex	wait/signal, Critical section 3 conditions
4. IPC	PC problem, RW, DP	Classical problems, shared memory, pipes
5. Deadlock	4 conditions, Banker	Prevention/Avoidance/Detection/Recovery
6. Memory	Paging, virtual memory	FIFO/LRU/Optimal, Thrashing, Demand paging

Chapter	Key Topics	Most Important Points
7. Disk	FCFS/SSTF/SCAN/C-SCAN	C-SCAN = uniform wait, SSTF = minimum seek
8. Linux	Commands, shell scripts	fork/exec/wait, chmod, grep, awk
9. Virtual	Hypervisors, VM types	Type1=bare metal, Type2=on host OS, JVM

IMPORTANT FORMULAS AND CONCEPTS

Process Scheduling Formulas

```
TAT (Turnaround Time) = Completion Time (CT) – Arrival Time (AT)
WT (Waiting Time) = TAT – Burst Time (BT)
Response Time = First CPU Time – Arrival Time
CPU Utilization = (CPU busy time / Total time) × 100%
Throughput = Number of processes completed / Time
```

Page Addressing

```
Logical Address = (Page Number p, Offset d)
Physical Address = Frame Number × Page Size + Offset
Number of pages = Process size / Page size
Number of bits for page offset = log■(Page size)
```

Memory Utilization

```
Internal Fragmentation = Partition size – Process size
External Fragmentation = Total free memory not usable (non-contiguous)
```

Disk Seek

```
Total head movement = Sum of |current - next| for each move
Average access time = Seek time + Rotational latency + Transfer time
```

TECHNICAL GLOSSARY

Term	Definition
Aging	Gradually increasing priority of waiting processes to prevent starvation
Atomic operation	Operation that cannot be interrupted (executes completely or not at all)

Term	Definition
Belady's Anomaly	FIFO: more frames → more page faults (paradox)
Blocking	Process suspended while waiting for resource/event
Bounded buffer	Buffer with limited capacity (Producer-Consumer problem)
Busy waiting	Process repeatedly checks condition in a loop (wastes CPU)
Cache	Fast storage between CPU and RAM
Compaction	Moving processes in memory to combine free spaces
Context switch	Saving current process state and loading another
Critical section	Code that accesses shared resource
Daemon	Background process (no user interaction)
Deadlock	All processes permanently blocked waiting for each other
Demand paging	Pages loaded only when needed
Dispatcher	OS component that gives CPU control to selected process
External fragmentation	Free memory exists but not contiguous
Frame	Fixed-size physical memory block
Hypervisor	Software that creates and manages VMs
Internal fragmentation	Wasted space inside allocated partition
IPC	Inter-Process Communication
Kernel	Core of OS, always in memory
LRU	Least Recently Used – page replacement policy
Mutex	Mutual exclusion lock (binary semaphore)
Page	Fixed-size logical memory block
Page fault	Required page not in RAM, must load from disk
Page table	Maps logical page numbers to physical frame numbers
PCB	Process Control Block – all info about a process
PID	Process ID – unique identifier
Preemption	Taking CPU from running process
Race condition	Result depends on execution order
RAID	Redundant Array of Independent Disks
Scheduling	Deciding which process gets CPU next
Semaphore	Integer variable for synchronization
Shell	Command interpreter
Starvation	Process waits indefinitely while others run
Swapping	Moving process between RAM and disk
System call	Program's request to OS for service
Thread	Smallest execution unit within a process
Thrashing	Too many page faults, CPU busy swapping
Virtual memory	Using disk to extend apparent RAM size
VM	Virtual Machine

MOST IMPORTANT EXAM QUESTIONS

Short Answer (2-3 marks)

1. Define Operating System and list its functions.
2. What is a system call? Give 4 examples.
3. Define: Process, Thread, PCB.
4. What are the 5 states of a process? Draw the state diagram.
5. Differentiate preemptive and non-preemptive scheduling.
6. Define: TAT, WT, Burst Time, Arrival Time.
7. What is mutual exclusion? State 3 requirements for critical section.
8. What is a semaphore? Explain wait() and signal().
9. State Coffman's 4 conditions for deadlock.
10. What is paging? Explain page table.
11. What is virtual memory? What is demand paging?
12. What is thrashing? How to prevent it?
13. Differentiate internal and external fragmentation.
14. What is FIFO page replacement? Explain Belady's Anomaly.
15. Differentiate SCAN and C-SCAN disk scheduling.
16. What is a shell? What is a shebang line?
17. What is virtualization? Differentiate Type 1 and Type 2 hypervisor.
18. What is JVM? Why is it called virtual machine?
19. What is RAID? Name 3 levels.
20. What is starvation? How is it different from deadlock?

Long Answer / Numerical (5-7 marks)

1. Explain all 4 CPU scheduling algorithms (FCFS, SJF, RR, Priority) with examples and Gantt charts.
2. Solve FCFS/SJF/RR scheduling numericals and calculate avg WT and TAT.
3. Explain Banker's Algorithm with an example. Find safe sequence.
4. Explain FIFO, LRU, Optimal page replacement with examples. Calculate page faults.
5. Explain SCAN and C-SCAN disk scheduling with example. Calculate total head movement.
6. Explain Producer-Consumer Problem. Give semaphore-based solution.
7. Explain Dining Philosopher Problem and its solutions.
8. Explain Reader-Writer Problem with semaphore solution.
9. Explain deadlock prevention, avoidance, detection, and recovery.
10. Explain paging with address translation. Show numerical example.
11. Write shell scripts for: (a) factorial, (b) prime check, (c) palindrome.
12. Write C program for fork() to create N child processes.
13. Explain virtualization. Differentiate Type 1 and Type 2 hypervisor.
14. Explain Peterson's solution for critical section.
15. Explain memory allocation strategies (First Fit, Best Fit, Worst Fit) with example.

Numerical Practice Problems

Scheduling Numerical:

Problem: Process P1 (AT=0, BT=8), P2 (AT=1, BT=4), P3 (AT=2, BT=9), P4 (AT=3, BT=5).

Find avg WT and TAT using: (a) FCFS (b) SJF (c) RR with quantum=2.

Page Replacement Numerical:

Problem: Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 with 3 frames.

Find page faults using FIFO, LRU, Optimal.

Disk Scheduling Numerical:

Problem: Request queue: 176,79,34,60,92,11,41,114. Initial head=50, moving toward higher.

Calculate head movement for FCFS, SSTF, SCAN, C-SCAN.

Banker's Algorithm:

Problem: 5 processes, 3 resources (A=10, B=5, C=7).

Given Allocation and Max matrices, find if system is in safe state.

End of GTU Operating Systems – Complete Study Notes

Subject Code: BE04000221 | Prepared for Semester 4 B.E. Students

Best of luck for your exams! ■